Agentic AI for Business and FinTech (FTEC5660)
Individual Homework 02_Part1 Due date: 2/27, midnight
Name: YANG Jinyue Student ID:1155242966

1. Introduction

This project aims to develop an autonomous agent that verifies candidate information from CVs against public social networks to assess their authenticity. The system can effectively deal with real-world interference noise, including spelling errors, location errors, and semantic variations such as abbreviations, and accurately identify fraudulent content.

2. System Architecture & Key Decisions

Based on the ReAct (Reasoning + Action) loop, develop an automated KYC and resume verification system with a pipeline: Data Extraction → Multi-platform Search → Detail Checking → Scoring.

Data Extraction

• Architecture**:**

Uses Google Gemini 2.0 Flash (Temperature = 0) to parse unstructured CV text and extract the Candidate Name, Target Companies, and Location.

• Design Decision:

By zero-shot prompt technology instead of hard-coded rules, it processes different resume formats to make sure the large language model understands candidate's professional identity before the search.

Tool and Multi-platform Search

• Architecture:

Manage interaction with the external MCP server via a Python execution loop, routing queries to search_linkedin_people or Facebook tools.

• Design Decision:

When the agent searched for exact strings like invalid locations "Singapore, Kowloon", the API returned no results. The system uses a multi-stage search strategy. If the initial name and location search fails, the agent will abandon the location parameter and search again using only the candidate's name, which ensures that real candidates are not overlooked because of minor spelling errors.

• Design Decision:

During testing the loop, the Gemini model sometimes misunderstood the data type, outputting the person_id as a string (e.g., "456") instead of the integer required by the MCP tool. A Regex filter (re.findall(r'\d+', ...)) inside the Python execution loop is added in the system to automatically extract the numbers and makes them integers.

Detail Checking

• Architecture:

This is the core reasoning part and the agent compares retrieved profiles with CVs.

• Design Decision:

By updating the system prompt to enforce Semantic Matching, the LLM is told to allow for business abbreviations and parent/subsidiary company names to prevent valid CVs from being falsely rejected.

Scoring

• Architecture:

This part standardizes the output format for the evaluation script. It lets the LLM generate a structured JSON object with a numerical score (0.0 to 1.0) and a reasoning string.

• Design Decision:

For "same-name" false positives, I gave agent a strict rule: if there is zero overlap in key companies, it must be marked as a fake CV using a real person's identity, resulting in an immediate score of 0.0.

• Design Decision:

LLMs struggle with iterative arithmetic and state-tracking, which often leads to the over-penalization of valid CVs. With a Holistic Scoring Rubri instead of doing math, the agent classifies the CV into discrete tiers (1.0 for Pass, 0.5 for Suspicious, 0.0 for Fatal Red Flag) based on the severity of the mismatch. This avoids arithmetic hallucinations.

3. Agent Workflow and Tool Usage Strategy

```
[ Input: CV (PDF) ]
        ↓
[ 1: LLM Extraction ] ──→ Extracts: Name, Location, Companies
        ↓
[ 2: MCP Tool Search ] ──→ search_linkedin_people(Name, Location)
        │
        ├── If No Results ──→ Fallback: search_linkedin_people(Name Only)
        ├── If Still Empty ──→ Fallback: Try fuzzy=True / Facebook API
        ↓
[ 3: Profile Retrieval ] ──→ Pull full profile via get_linkedin_profile(ID)
        ↓
[ 4: Semantic Verification & JSON Output ]
        │
        ├── Check 1: Apply abbreviation rules (e.g., PwC == PricewaterhouseCoopers)
        ├── Check 2: Evaluate Timeline & Company overlap
        (Decsion)
        ├── Is Company History a Total Fabrication? ──→ Output: Score 0.0 (Fail)
        ├── Do core claims match the profile? ──→ Output: Score 1.0 (Pass)
```

4. Sample Verification Results

The system achieved 100% accuracy on the provided test set.

```
...
  [1/5] Processing CV_1.pdf...
    ✅ Decision: I found a LinkedIn profile for John Smith in Singapore. The CV states he worked at ByteDance as an Engineer from 2020 - Present. The LinkedIn profile confirms th
    ⭐ Score: 1.0

  [2/5] Processing CV_2.pdf...
    ✅ Decision: I found a profile for Minh Pham in Hong Kong. The profile lists experience at BCG (Boston Consulting Group) and Tencent, which matches the CV. The job titles are
    ⭐ Score: 1.0

  [3/5] Processing CV_3.pdf...
    ✅ Decision: I found a profile for Wei Zhang in Munich. The CV states that Wei Zhang worked as an Engineer at PwC from 2013 to Present. PwC is an abbreviation for Pricewaterh
    ⭐ Score: 1.0

  [4/5] Processing CV_4.pdf...
    ✅ Decision: I searched for Rahul Sharma in Singapore. I did not find a profile that matches the CV. The CV states that Rahul Sharma worked as a Senior Engineer at Microsoft
    ⭐ Score: 0.0

  [5/5] Processing CV_5.pdf...
    ✅ Decision: I am unable to verify the CV. The dates for DataForge and UrbanFlow overlap, which is impossible. Also, the dates for DataForge are in the future.
    ⭐ Score: 0.0

  Final Scores List: [1.0, 1.0, 1.0, 0.0, 0.0]
```

```
# scores = ... # Your code should generate this list [0.2, 0.3, 0.4, 0.5, 0.6]
groundtruth = [1, 1, 1, 0, 0] # Do not modify

result = evaluate(scores, groundtruth)
print(result)
```

```
{'decisions': [1, 1, 1, 0, 0], 'correct': 5, 'total': 5, 'final_score': 1.0}
```

Figure: Execution Log from Google Colab